

RAJALAKSHMI ENGINEERING COLLEGE
AN AUTONOMOUS INSTITUTION
AFFILIATED TO ANNA UNIVERSITY
RAJALAKSHMI NAGAR, THANDALAM,
CHENNAI-602105



RAJALAKSHMI
ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

DEPARTMENT OF COMPUTER SCIENCE
AND ENGINEERING

CS26321 COMPILER DESIGN LABORATORY
ACADEMIC YEAR: 2025-2026 (EVEN)

INDEX

Exp No	Date	Name of the Experiment	Mark/ Viva	Signature
1		Develop A Simple C Program To Demonstrate A Basic String Operations		
2		Develop A C Program To Analyser A Given C Code Snippet And Recognize Different Tokens, Including Keyword, Identifiers, Operator, Delimiter And Special Symbols		
3		Write a LEX program to recognize valid arithmetic expressions. Identifiers in the expression could be only integers and operators could be + and *. Count the identifiers & operators present and print them separately.		
4		Write a LEX program to eliminate comment lines in a C program and copy the resulting program into a separate file.		
5		Write a YACC program to recognize all strings for which starts with n number of "a" followed by n number of "b".		
6		Write a YACC program to recognize valid identifiers, operators and keywords in the given text (C program) file.		
7		Implementation of calculator using LEX and YACC.		
8		Convert the BNF rules into YACC form and write code to generate abstract syntax trees.		
9		Write a scheme expression that evaluates the polynomial. Write $5 * (4.5 - 8.5) + 77$ as a scheme expression, and find its value. Define a function middle that takes five numbers as argument and returns the middle of the five.		
10		Intro to Flex and Bison: Modify the scanner and parser so that terminating a statement with ";b" instead of ";" results in the output being printed in binary.		
11		Write a recursive descent parser for the CFG language and implement it using LLVM		
12		Write a LR parser for the CFG language and implement it in the using LLVM		

Hardware requirements:PC

Software requirements:C Compiler, Editplus (FLEX)

Reference annexure 1 for installation

EXP NO:

DATE:

DEVELOP A SIMPLE C PROGRAM TO DEMONSTRATE A BASIC STRING OPERATIONS

Questions

1. Write a C program to find the length of a string without using strlen().
2. Write a C program to copy one string to another without using strcpy().
3. Write a C program to reverse a string without using inbuilt functions.
4. Write a C program to count the number of vowels and consonants in a string.
5. Write a C program to find the frequency of each character in a string.
6. Write a C program to remove all spaces from a string.
7. Write a C program to count digits, alphabets, and special characters in a string.

AIM:

ALGORITHM:

PROGRAM:

OUTPUT:

AIM:

ALGORITHM:

PROGRAM:

OUTPUT:

AIM:

ALGORITHM:

PROGRAM:

OUTPUT:

AIM:

ALGORITHM:

PROGRAM:

OUTPUT:

AIM:

ALGORITHM:

PROGRAM:

OUTPUT:

AIM:

ALGORITHM:

PROGRAM:

OUTPUT:

AIM:

ALGORITHM:

PROGRAM:

OUTPUT:

Implementation	
Output/Signature	

RESULT:

EXP NO:

DATE:

**DEVELOP A C PROGRAM TO ANALYSE A GIVEN C CODE SNIPPET AND
RECOGNIZE DIFFERENT TOKENS, INCLUDING KEYWORD, IDENTIFIERS,
OPERATOR, DELIMITER AND SPECIAL SYMBOLS**

AIM:

To develop a C program that analyses a given C code snippet and recognizes different tokens, including keywords, identifiers, operators, delimiter and special symbols.

ALGORITHM:

- **Start**
- Take a C code snippet as input from the user or a file.
- Initialize necessary arrays and variables for keywords, identifiers, operators, and special symbols.
- Tokenize the input string using spaces, newlines, and other delimiters.
- For each token:
 - Check if it is a **keyword** (compare with a predefined list of C keywords).
 - Check if it is an **identifier** (valid variable/function name that doesn't match a keyword).
 - Check if it is an **operator** (e.g., +, -, *, /, ==, &&).
 - Check if it is a **special symbol** (e.g., {, }, (,), ;, ,).
- Print the categorized tokens.
- **End**

PROGRAM:

OUTPUT:

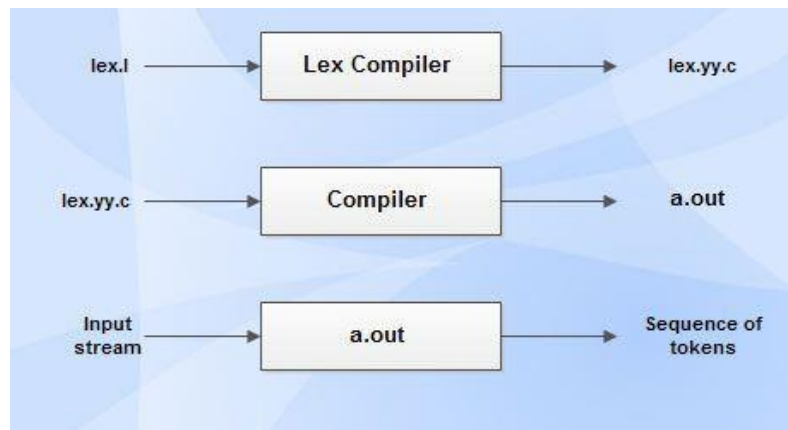
Implementation	
Output/Signature	

RESULT :

STUDY OF LEX TOOL

LEX:

Lex is a tool in lexical analysis phase to recognize tokens using regular expression. Lex tool itself is a lex compiler.



- lex.l is an input file written in a language which describes the generation of lexical analyzer. The lex compiler transforms lex.l to a C program known as lex.yy.c.
- lex.yy.c is compiled by the C compiler to a file called a.out.
- The output of C compiler is the working lexical analyzer which takes stream of input characters and produces a stream of tokens.
- yylval is a global variable which is shared by lexical analyzer and parser to return the name and an attribute value of token.
- The attribute value can be numeric code, pointer to symbol table or nothing.
- Another tool for lexical analyzer generation is Flex.

STRUCTURE OF LEX PROGRAMS:

Lex program will be in following form

declarations

%%

translation rules

%%

auxiliary functions

Declarations: This section includes declaration of variables, constants and regular definitions.

Translation rules: It contains regular expressions and code segments.

Form : Pattern {Action}

Pattern is a regular expression or regular definition.

Action refers to segments of code.

Patterns for tokens in the grammar

- *digit* → [0-9]
- digits* → *digit*⁺
- number* → *digits* (.*digits*)? (E [+]*digits*)?
- letter* → [A-Za-z]
- id* → *letter* (*letter* | *digit*)^{*}
- if* → if
- then* → then
- else* → else
- relop* → < | > | <= | >= | = | <>
- *ws* → (blank | tab | newline)⁺

```
%{ LT, LE, EQ, NE, GT, GE, IF,
    THEN, ELSE, ID, NUMBER, RELOP
}%
delim    [ \t\n]
ws       {delim}+
letter   [A-Za-z]
digit    [0-9]
id       {letter}({letter} | {digit})*
number   {digit}+(\.{digit})?(E[+]?{digit})+?
%%
{ws}     {}
if        { return(IF); }
then      { return(THEN); }
else      { return(ELSE); }
```

Auxiliary functions: This section holds additional functions which are used in actions. These functions are compiled separately and loaded with lexical analyzer. Lexical analyzer produced by lex starts its process by reading one character at a time until a valid match for a pattern is found. Once a match is found, the associated action takes place to produce token. The token is then given to parser for further processing.

CONFLICT RESOLUTION IN LEX:

Conflict arises when several prefixes of input matches one or more patterns. This can be resolved by the following:

- Always prefer a longer prefix than a shorter prefix.
- If two or more patterns are matched for the longest prefix, then the first pattern listed in lex program is preferred

yylex()

a function implementing the lexical analyzer and returning the token matched

yytext

a global pointer variable pointing to the lexeme matched

yylen

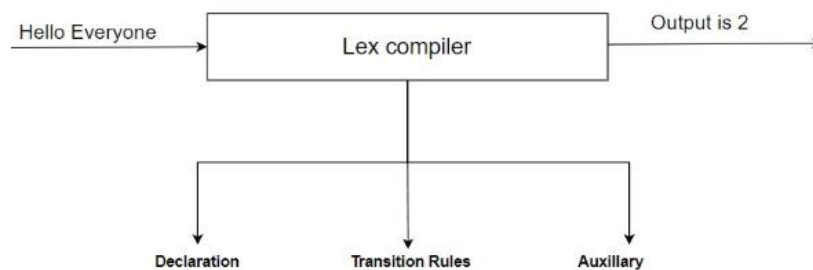
a global variable giving the length of the lexeme matched

yyval

an external global variable storing the attribute of the token

yywrap:

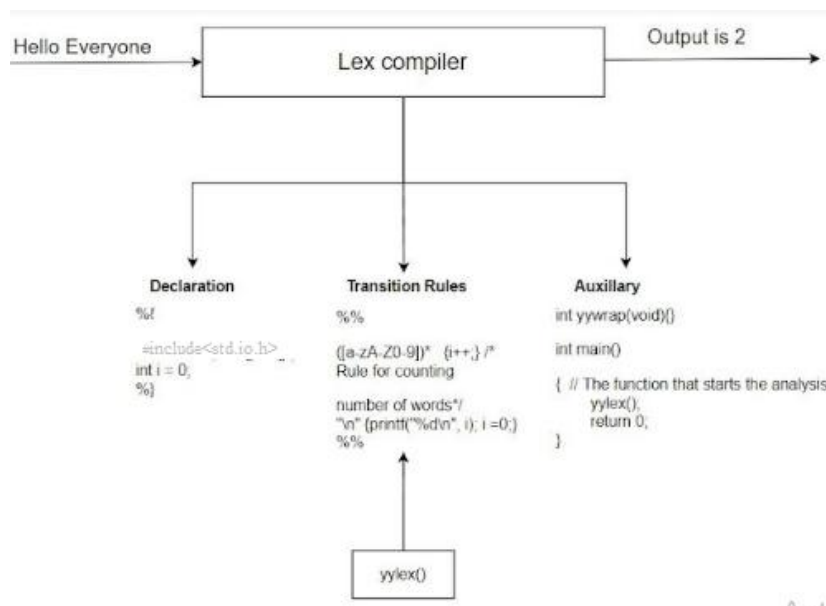
Function yywrap is called by lex when input is exhausted. Return 1 if you are done or 0 if more processing is required.



Declaration-Has header files and initialization of variables

Translation rules-write the rules for counting the words

Auxillary- has yylex() that calls the translation rules



Simple lex program:

```
/*lex program to count number of words*/
%{
#include<stdio.h>
#include<string.h>

int i = 0;
%}

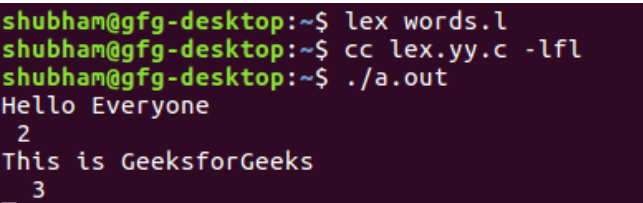
/* Rules Section*/
%%
([a-zA-Z0-9])* {i++;} /* Rule for counting
                    number of words*/

"\n" {printf("%d\n", i); i = 0;}
%%

int yywrap(void){}

int main()
{
    // The function that starts the analysis
    yylex();

    return 0;
}
```



```
shubham@gfg-desktop:~$ lex words.l
shubham@gfg-desktop:~$ cc lex.yy.c -lfl
shubham@gfg-desktop:~$ ./a.out
Hello Everyone
2
This is GeeksforGeeks
3
```


EXP NO:

DATE:

Write A Lex Program To Recognize Valid Arithmetic Expressions. Identifiers In The Expression Could Be Only Integers And Operators Could Be + And *. Count The Identifiers & Operators Present And Print Them Separately.

AIM:

To write a LEX program to recognize a valid arithmetic expression where identifiers are integers only and operators are + and *, and to count and display the number of identifiers and operators separately.

ALGORITHM:

- Start.
- Read the arithmetic expression from the user.
- Initialize counters for identifiers and operators to zero.
- Scan the expression from left to right.
- If a number (integer) is encountered, count it as one identifier.
- If an operator (+ or *) is encountered, count it as one operator.
- Ignore blank spaces and newline characters.
- Continue scanning until the end of the expression.
- Display the total count of identifiers and operators separately.
- Stop.

PROGRAM:

OUTPUT:

Implementation	
Output/Signature	

RESULT:

EXP NO:

DATE:

WRITE A LEX PROGRAM TO ELIMINATE COMMENT LINES IN A C PROGRAM AND COPY THE RESULTING PROGRAM INTO A SEPARATE FILE

AIM:

To design and implement a lex program to eliminate comment lines in a c program and copy the resulting program into a separate file.

ALGORITHM :

- Start the program.
- Open the input C source file.
- Create and open a separate output file to store the comment-free program.
- Read the input file line by line using LEX rules.
- Identify and remove:
 - Single-line comments (`// ...`)
 - Multi-line comments (`/* ... */`)
- Copy all non-comment program statements into the output file.
- Ignore the comment lines completely.
- Continue scanning until the end of the input file.
- Close both input and output files.
- Stop the program.

PROGRAM:

OUTPUT :

Implementation	
Output/Signature	

RESULT:

STUDY OF YACC TOOL

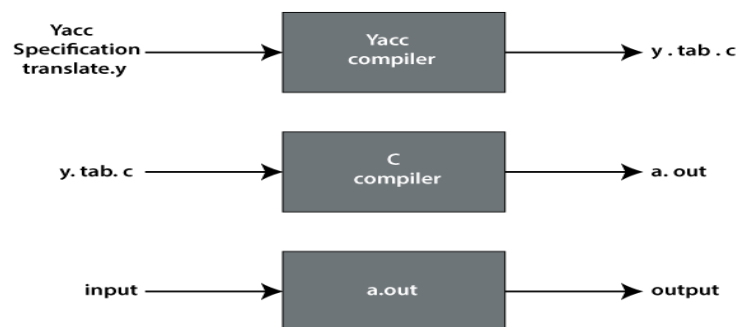
YACC TOOL:

YACC is known as Yet Another Compiler Compiler. It is used to produce the source code of the syntactic analyzer of the language produced by LALR (1) grammar. The input of YACC is the rule or grammar, and the output is a C program. If we have a file `translate.y` that consists of YACC specification, then the UNIX system command is:

YACC `translate.y`

This command converts the file `translate.y` into a C file `y.tab.c`. It represents an LALR parser prepared in C with some other user's prepared C routines. By compiling `y.tab.c` along with the `ly` library, we will get the desired object program `a.out` that performs the operation defined by the original YACC program.

The construction of translation using YACC is illustrated in the figure below:



STRUCTURE OF YACC:

Declarations

%%

Translation rules

%%

Supporting C rules

C declarations	<pre>%{\n#include <stdio.h>\n}</pre>
yacc declarations	<pre>%token NAME NUMBER\n%%</pre>
Grammar rules	<pre>statement: NAME '=' expression\n expression { printf("= %d\\n", \$1); }\n ;\n\nexpression: expression '+' NUMBER { \$\$ = \$1 + \$3; }\n expression '-' NUMBER { \$\$ = \$1 - \$3; }\n NUMBER { \$\$ = \$1; }\n ;\n\n%%</pre>
Additional C code	<pre>int yyerror(char *s)\n{\n fprintf(stderr, "%s\\n", s);\n return 0;\n}\n\nint main(void)\n{\n yyparse();\n return 0;\n}</pre>

Declarations Part: This part of YACC has two sections; both are optional. The first section has ordinary C declarations, which is delimited by %{ and %}. Any temporary variable used by the second and third sections will be kept in this part. Declaration of grammar tokens also comes in the declaration part. This part defined the tokens that can be used in the later parts of a YACC specification.

```
%{\n#include <stdio.h>\n#include <stdlib.h>\n%}\n%token ID NUM\n%start expr
```

Terminal

Start Symbol

Translation Rule Part: After the first %% pair in the YACC specification part, we place the translation rules. Every rule has a grammar production and the associated semantic action. A set of productions:

$$\langle \text{head} \rangle \Rightarrow \langle \text{body} \rangle_1 \mid \langle \text{body} \rangle_2 \mid \dots \mid \langle \text{body} \rangle_n$$

would be written in YACC as

```
<head> : <body>1    {<semantic action>1}\n      | <body>2    {<semantic action>2}\n      | ..... \n      | <body>n    {<semantic action>n}\n      ;
```


Grammar rule section

```
expr  : expr '+' term
      | term
      ;
term   : term '*' factor
      | factor
      ;
factor : '(' expr ')'
      | ID
      | NUM
```

In a YACC production, an unquoted string of letters and digits that are not considered tokens is treated as non-terminals.

The semantic action of YACC is a set of C statements. In a semantic action, the symbol \$\$ is considered to be an attribute value associated with the head's non-terminal. While \$i is considered as the value associated with the grammar production of the body. If we have left only with associated production, the semantic action will be performed. The value of \$\$ is computed in terms of \$i's by semantic action.

Supporting C–Rules: It is the last part of the YACC specification and should provide a lexical analyzer named **yylex()**. These produced tokens have the token's name and are associated with its attribute value. Whenever any token like DIGIT is returned, the returned token name should have been declared in the first part of the YACC specification.

The attribute value which is associated with a token will communicate to the parser through a variable called **yylval**. This variable is defined by a YACC.

Whenever YACC reports that there is a conflict in parsing-action, we should have to create and consult the file **y.output** to see why this conflict in the parsing-action has arisen and to see whether the conflict has been resolved smoothly or not.

EXP NO:

DATE:

Write a YACC program to recognize all strings for which starts with n number of "a" followed by n number of "b".

Problem Statement

Develop a YACC program to recognize and validate all strings of the form $a^n b^n$, where a string starts with n number of 'a' characters followed by n number of 'b' characters, with $n \geq 1$. The program should accept strings that satisfy this pattern and reject strings that do not conform to it.

AIM:

To write a **YACC program** to recognize all strings of the form $a^n b^n$, where the string starts with n number of 'a' characters followed by n number of 'b' characters.

ALGORITHM:

1. Start the program.
2. Read the input string from the user.
3. Initialize the YACC parser.
4. Define grammar rules to ensure:
5. The string contains equal numbers of 'a's and 'b's.
6. All 'a's appear before any 'b'.
7. Use recursive grammar rules to match one 'a' with one corresponding 'b'.
8. If the entire input string satisfies the grammar, accept the string.
9. If the string violates the grammar rules, reject the string.
10. Display whether the given string is valid or invalid.
11. Stop the program.

PROGRAM:

OUTPUT :

Implementation	
Output/Signature	

RESULT:

EXP NO:

DATE:

Problem Statement

Implement a simple calculator using LEX and YACC that evaluates arithmetic expressions involving integers using operators +, -, *, / and parentheses. The calculator should read an expression as input and display the evaluated result.

AIM:

To design and implement a calculator using LEX for lexical analysis and YACC for syntax analysis and evaluation of arithmetic expressions.

ALGORITHM:

Step 1: Start

Begin the program execution.

Step 2: Lexical Analysis using LEX

- Read the input arithmetic expression.
- Identify and return tokens such as:
 - Numbers (digits)
 - Operators (+, -, *, /)
 - Parentheses ((,))
- Ignore spaces and newline characters.
- Pass tokens to the YACC parser.

Step 3: Syntax Analysis using YACC

- Define grammar rules for arithmetic expressions.
- Handle operator precedence and associativity.
- Evaluate expressions based on grammar rules.

Step 4: Expression Evaluation

- Perform arithmetic operations according to grammar actions.
- Store intermediate results during parsing.

Step 5: Output

- Display the final evaluated result of the expression.

Step 6: Stop

Terminate the program after evaluation.

PROGRAM:

OUTPUT :

Implementation	
Output/Signature	

RESULT:

EXP NO:

DATE:

Convert the given BNF grammar rules into YACC form and implement a YACC program to generate an Abstract Syntax Tree (AST) for arithmetic expressions. The program should correctly represent operators and operands in the form of a tree structure.

AIM:

To convert BNF rules into YACC grammar and to generate an Abstract Syntax Tree (AST) for arithmetic expressions using syntax-directed translation in YACC.

ALGORITHM:

Step 1: Start

Begin the program execution.

Step 2: Define BNF Grammar

Specify the BNF rules for arithmetic expressions involving operators and operands.

Step 3: Convert BNF to YACC Form

- Rewrite BNF rules using YACC syntax.
- Specify operator precedence and associativity.
- Define tokens for numbers and operators.

Step 4: Design AST Node Structure

- Create a node structure containing:
 - Operator or operand
 - Left child pointer
 - Right child pointer

Step 5: Lexical Analysis

- Use LEX to identify numbers, operators, and parentheses.
- Pass tokens to the YACC parser.

Step 6: Syntax Analysis and AST Construction

- Attach semantic actions to YACC grammar rules.
- For each operator, create an AST node.
- Link operands as left and right children.

Step 7: Store AST Root

Save the root node of the generated Abstract Syntax Tree.

Step 8: Output

Traverse and display the AST (inorder / preorder).

Step 9: Stop

Terminate the program.

PROGRAM:

OUTPUT :

Implementation	
Output/Signature	

RESULT:

Annexure 1

Download editplus from the following link:

https://drive.google.com/file/d/0B9D4jOdpRzZHNTVraV9rX280R0E/view?resourcekey=0-wKxkIdA7Eqh0j2Jj_u87ow

Note: Make sure that C compiler is installed.

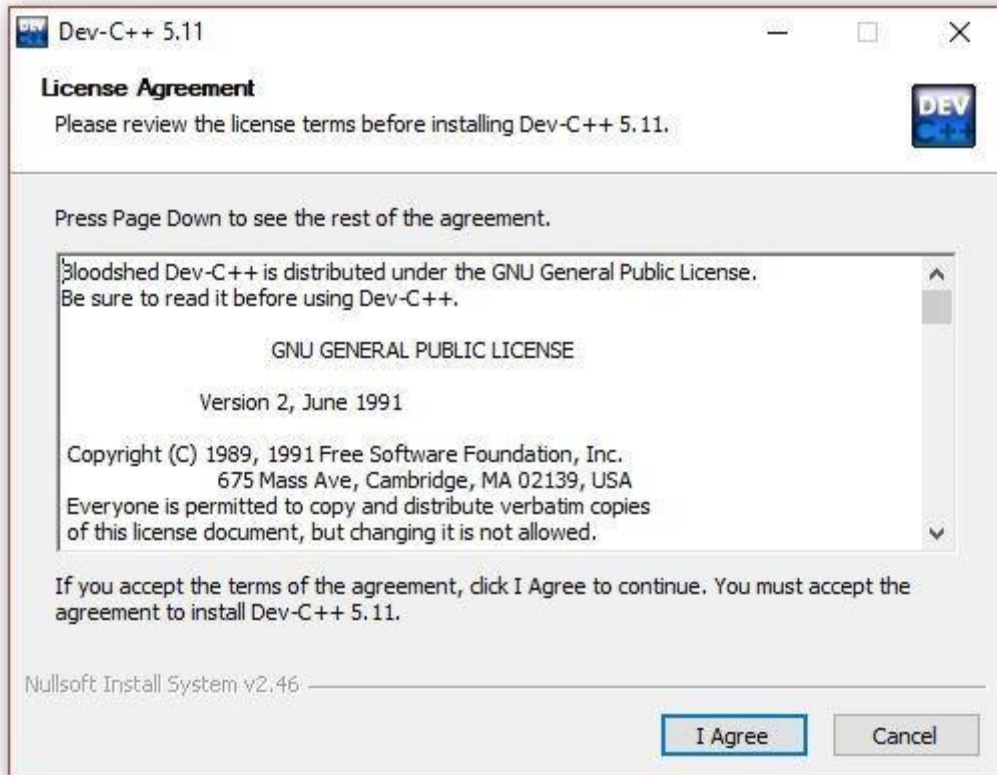
Install the setup:

After downloading the setup double click on it, it extracts the package from the setup and asks to select the language. So select the language as per your choice and press the ok button.

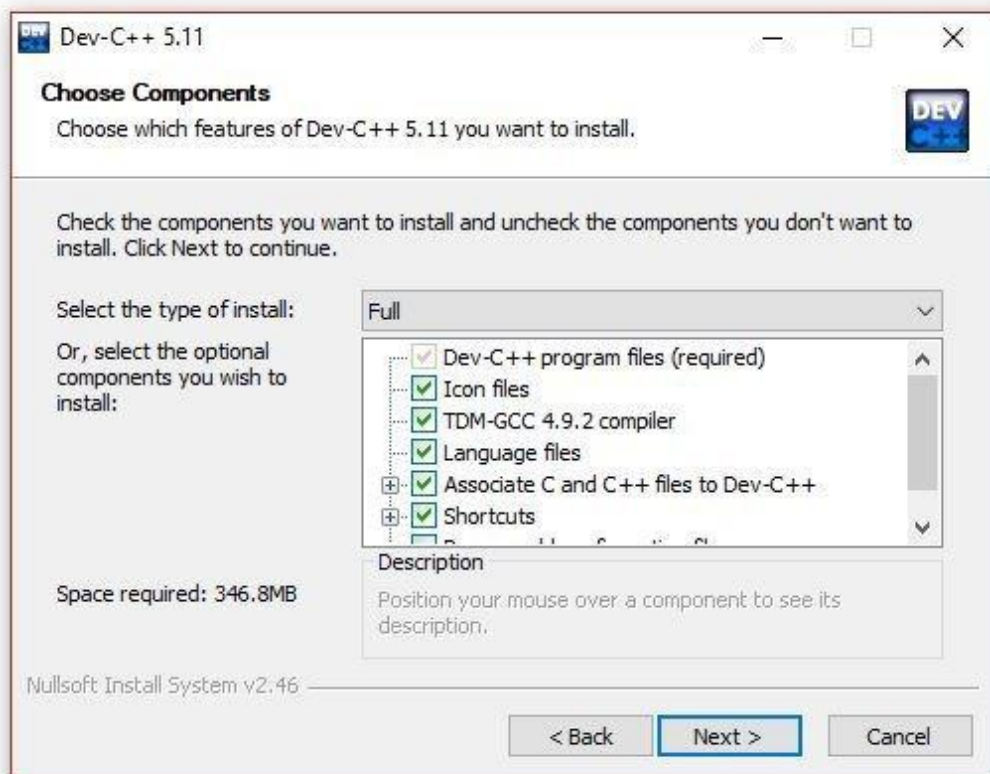
 Dev-Cpp 5.11 TDM-GCC 4.9.2 Setup	5/19/2015 10:51 AM	Application	49,252 KB
--	--------------------	-------------	-----------



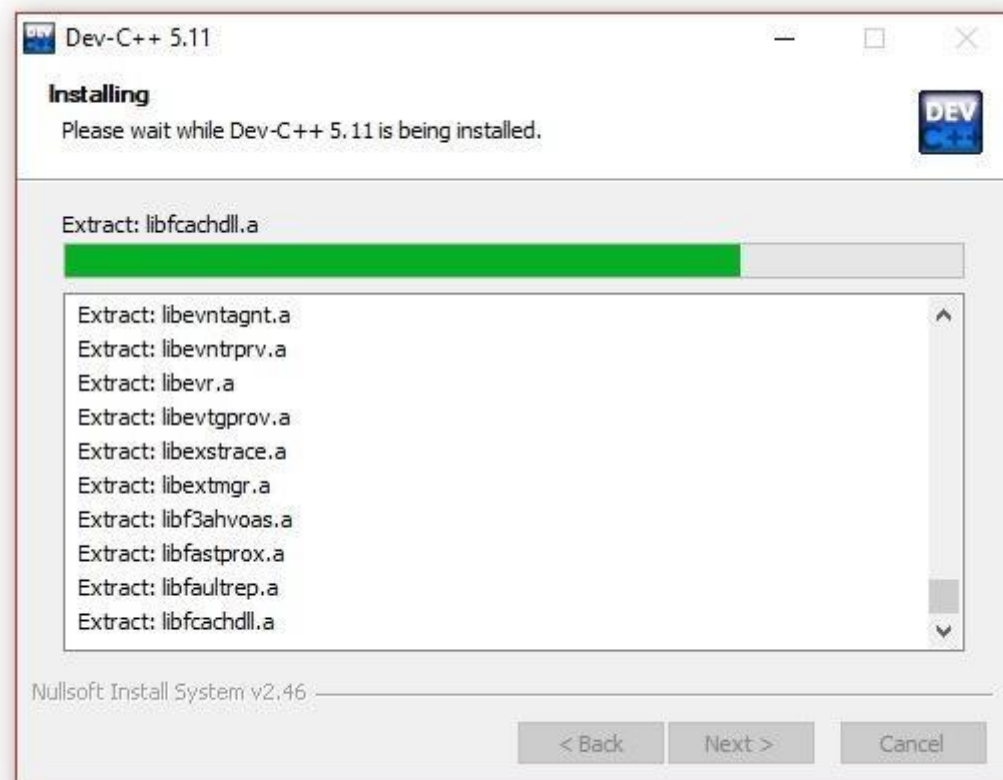
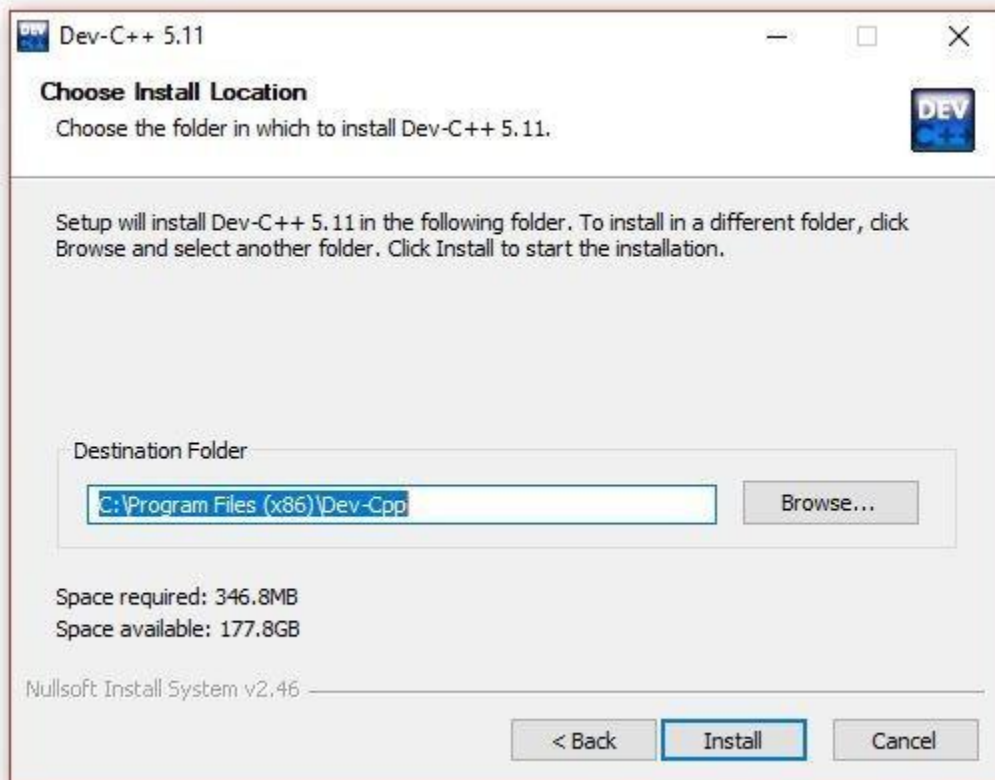
Accept the license agreement.



In the next prompt window, it will ask you to select the package you want to install. Here you don't need to do anything just press the Next button.



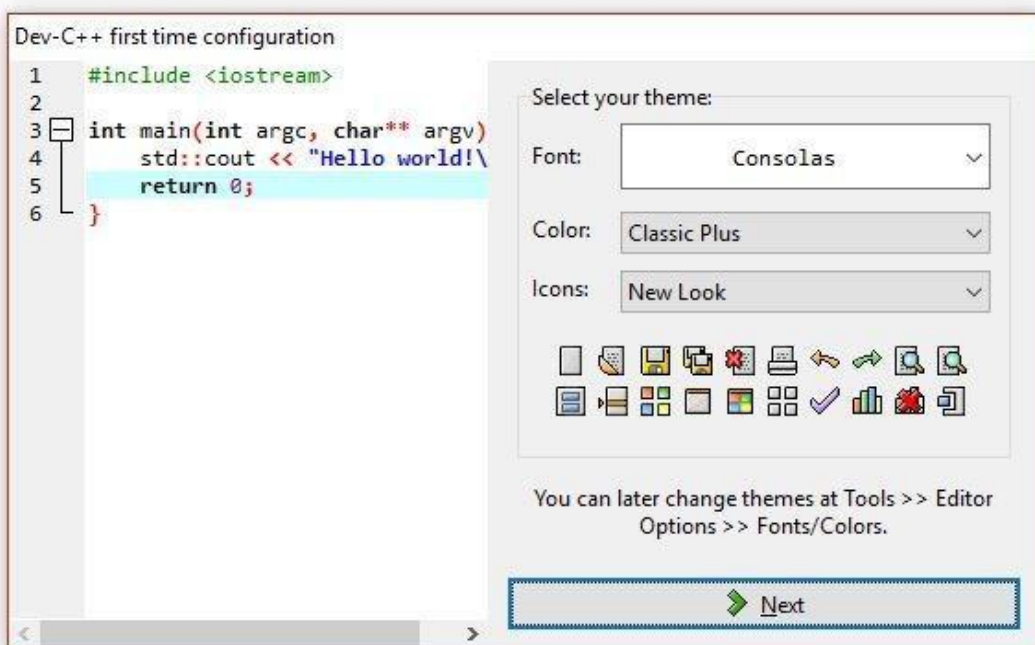
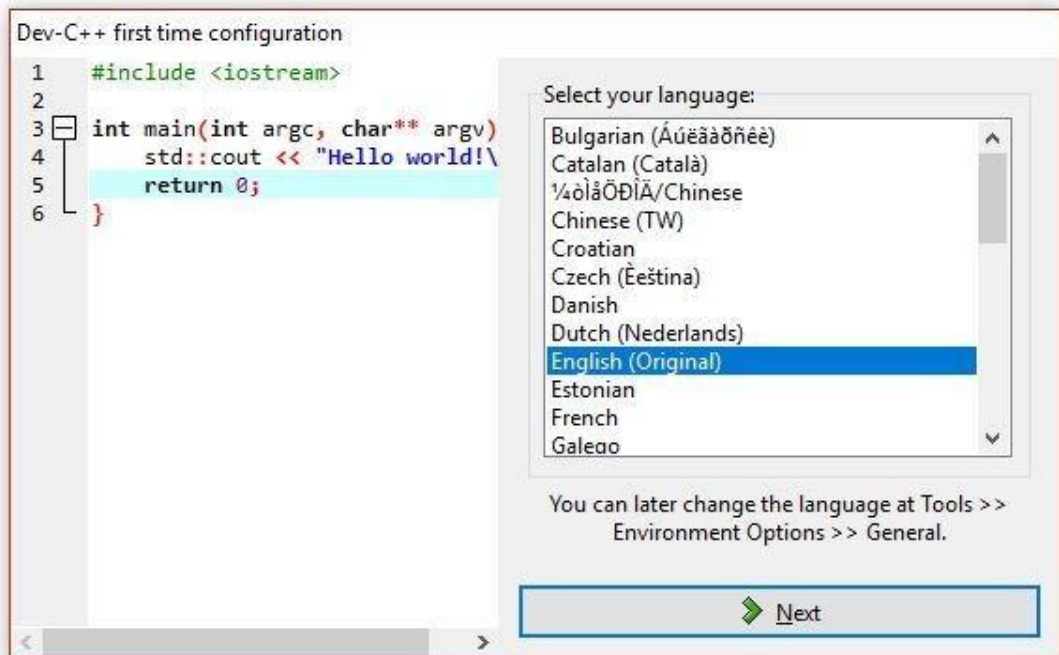
Select the destination folder (keep it to default) and press the install button. It will take a few minutes.

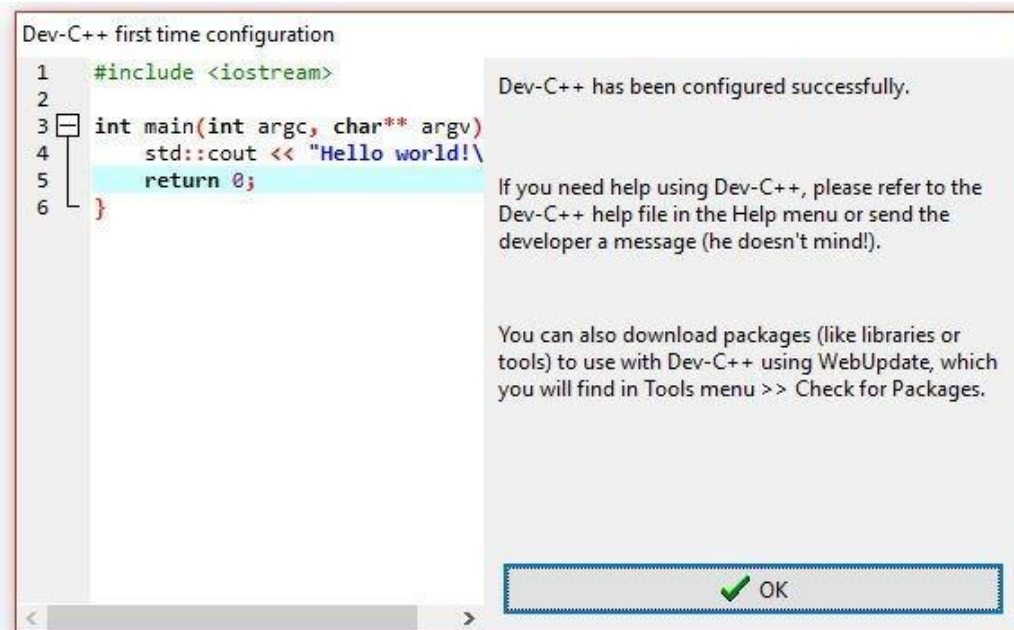


After completing the installation, Dev-C++ will prompt a window with a finish button. Just press the finish button.



After pressing the finish button it asks for language and fonts. If you want to change the language and fonts, then you can change.

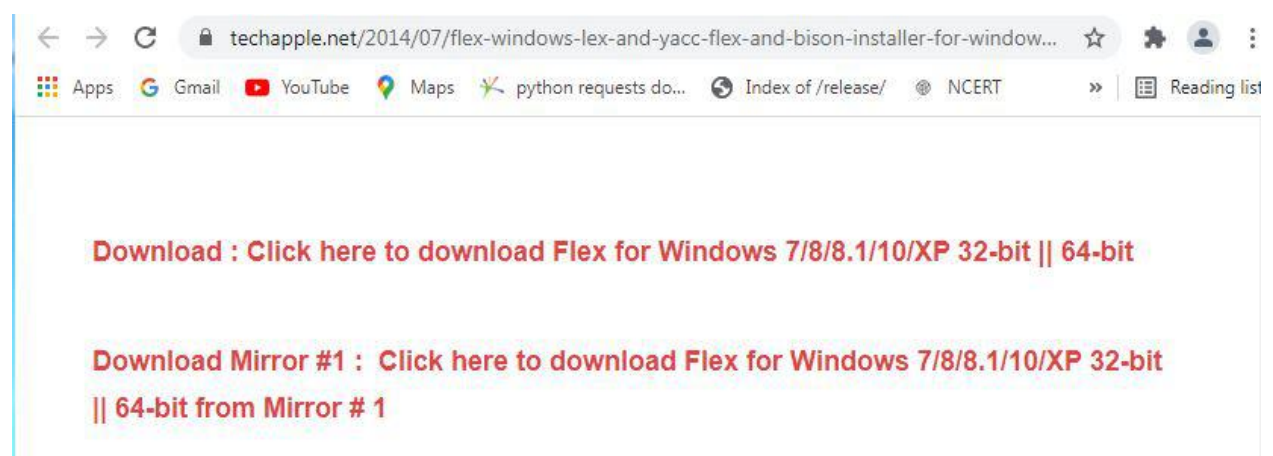




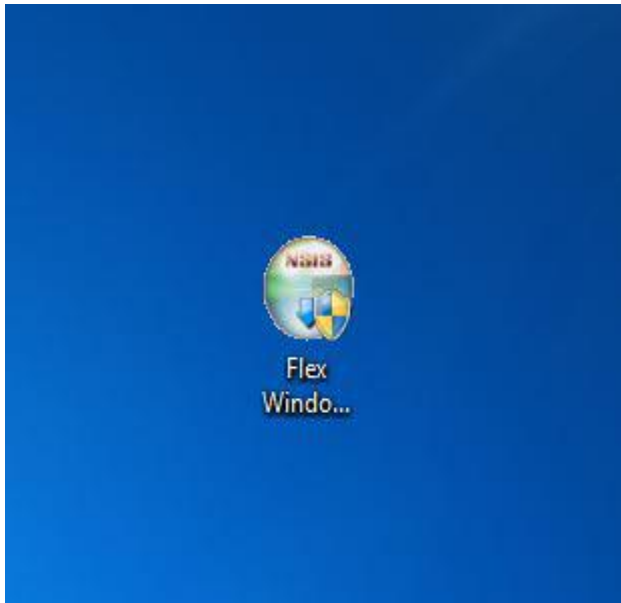
Flex - Lex and Yacc

Step 1 : Install from the above URL

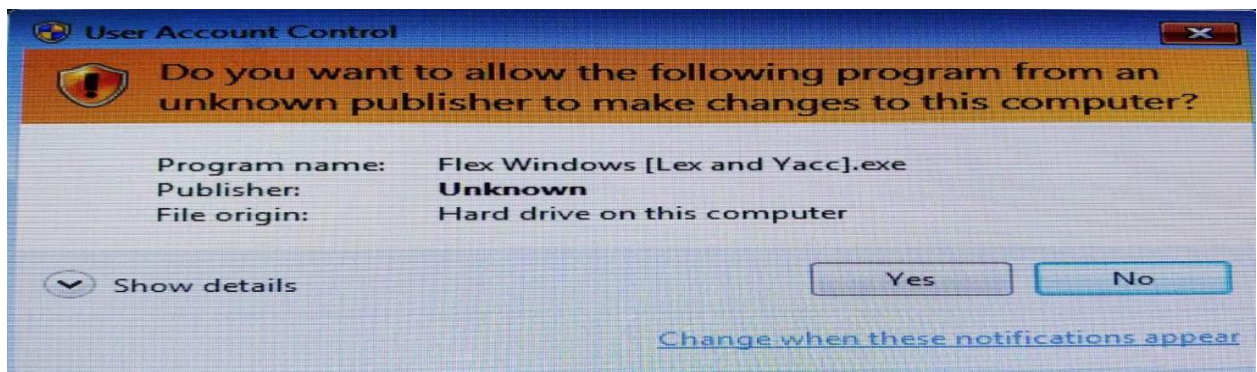
Step 2: On this page, all the features and minimum requirement of the system to install flex is given. Here the download link of the flex program for Windows XP, 7, 8, etc is given. Click on the download link, downloading of the executable file will start shortly. It is a small 30.19 MB file that will hardly take a minute.



Step 3: Now check for the executable file in downloads in your system and run it.



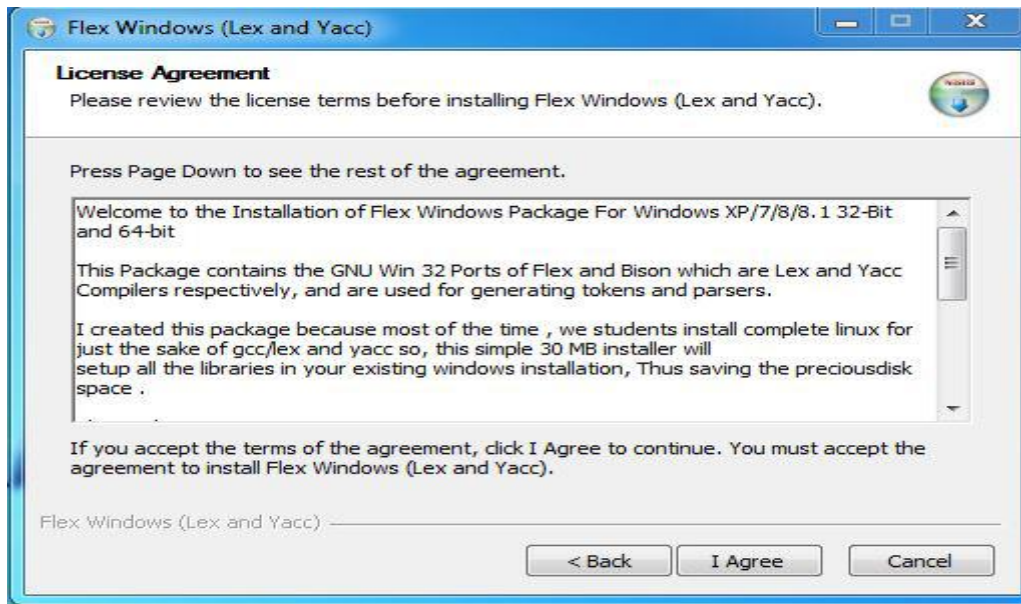
Step 4: It will prompt confirmation to make changes to your system. Click on Yes.



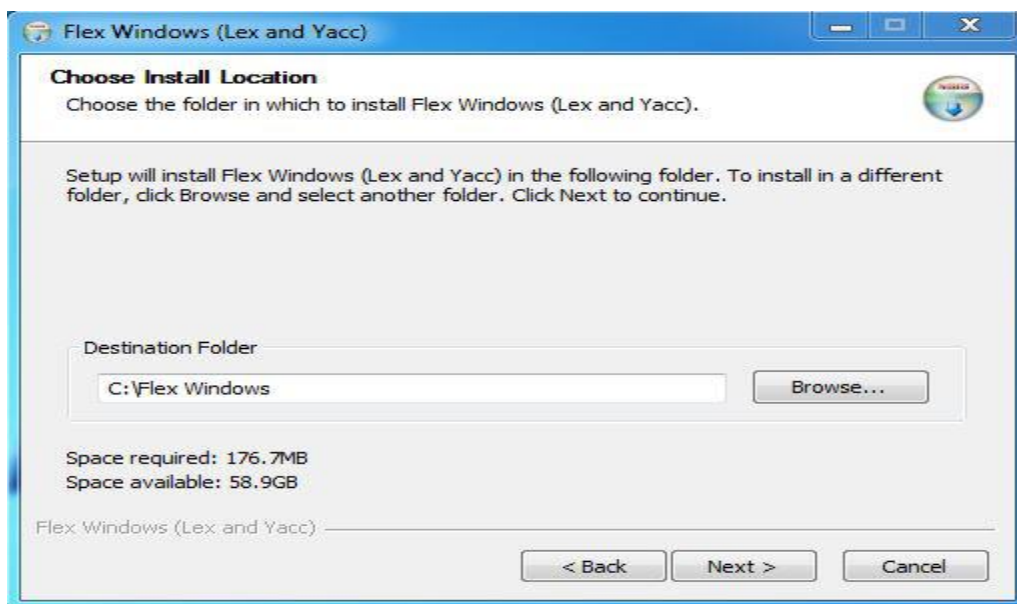
Step 5: Setup screen will appear, click on Next.



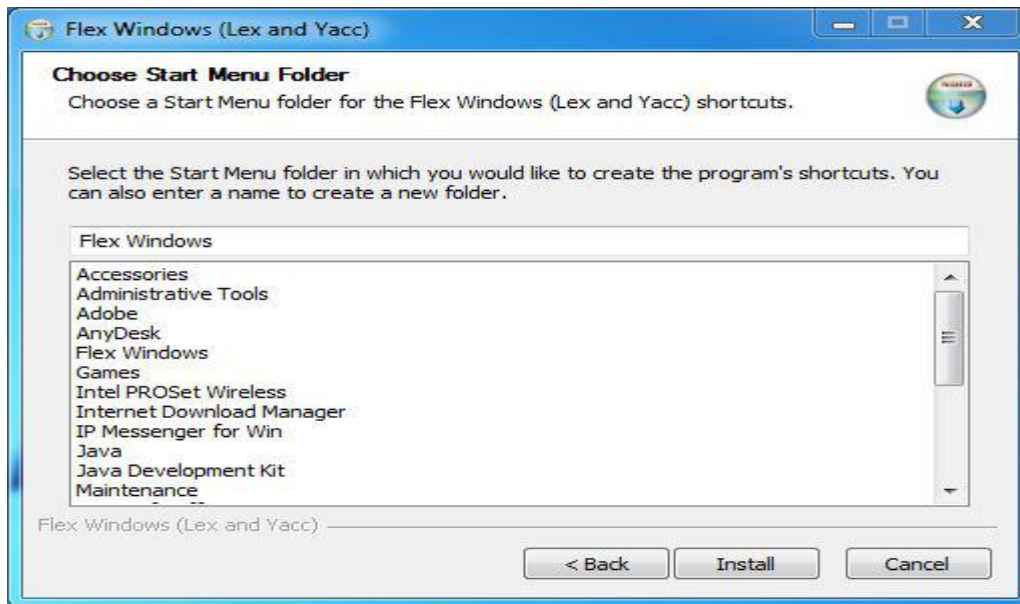
Step 6: The next screen will be of License Agreement, click on I Agree.



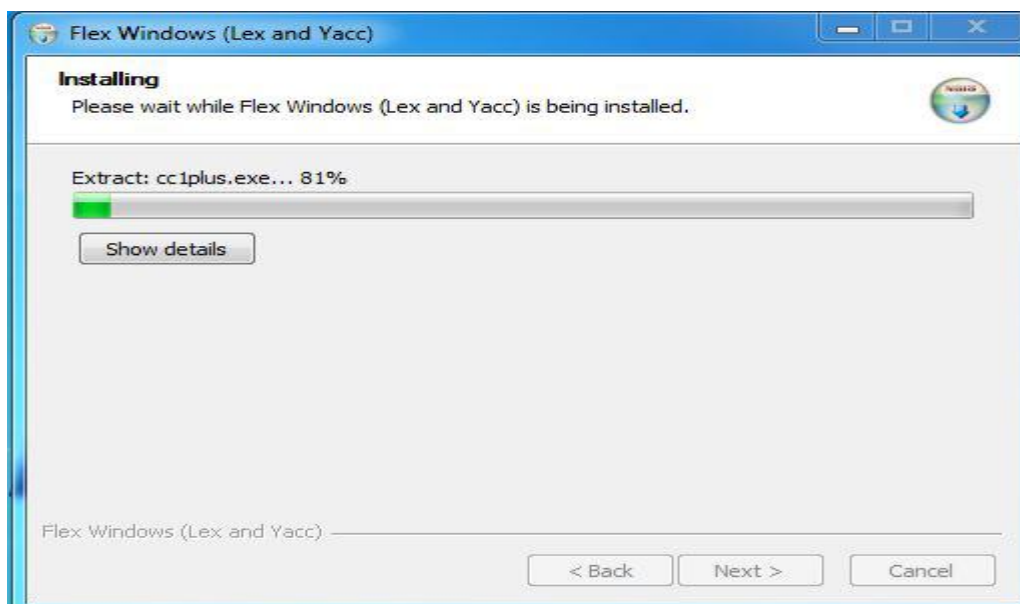
Step 7: The next screen will be of installing location so choose the drive which will have sufficient memory space for installation. It needed only a memory space of 176.7 MB.



Step 8: Next screen will be of choosing the Start menu folder so don't do anything just click on the Next Button.



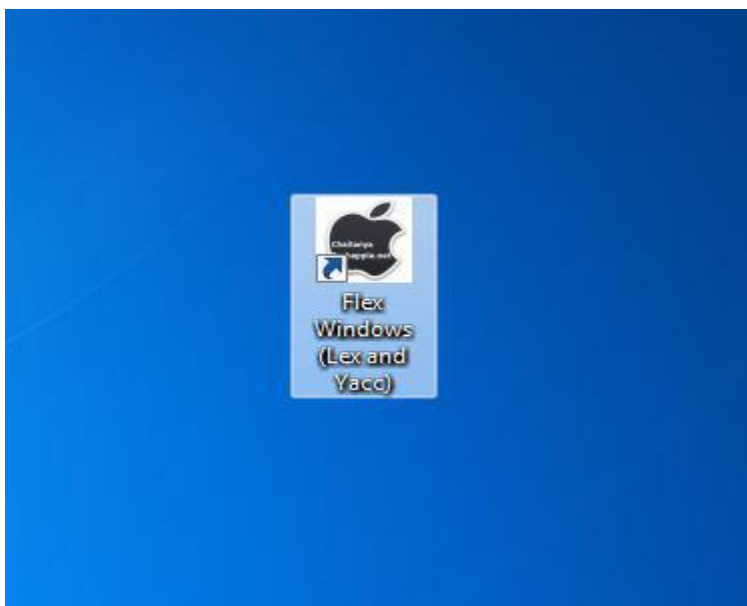
Step 9: After this installation process will start and will hardly take a minute to complete the installation.



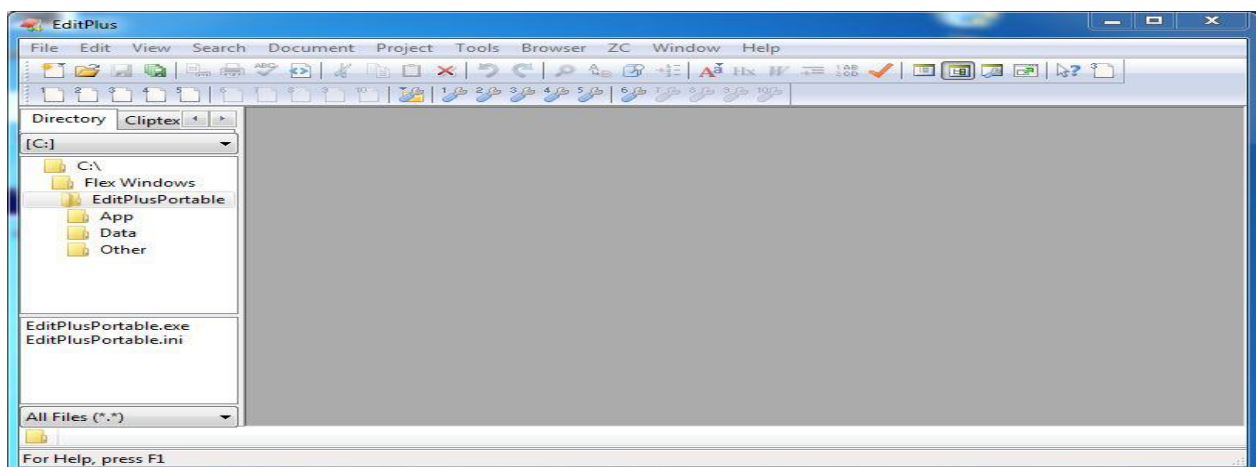
Step 10: Click on Finish after the installation process is complete. Keep the tick mark on the checkbox if you want to run Flex now if not then uncheck it.



Step 11: Flex Windows is successfully installed on the system and an icon is created on the desktop

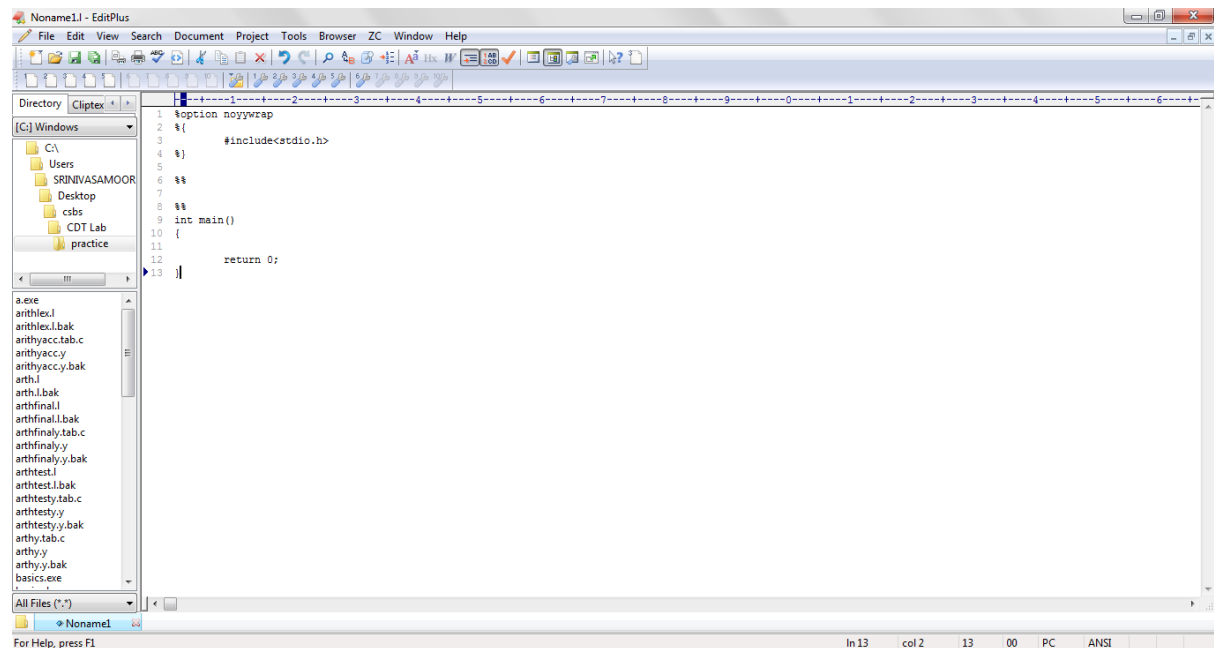


Step 12: Run the software and see the interface.



Flex is now installed on the system

To create a new file file>new>lex then the file would be Noname1.l then we can save and use lex file

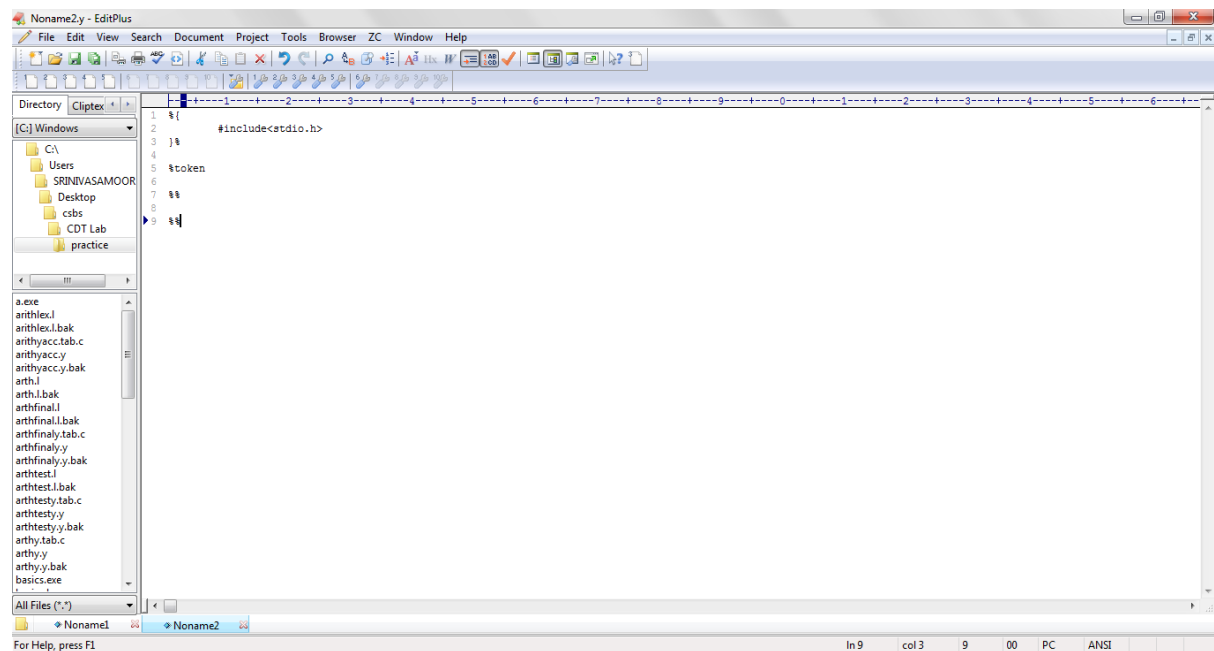


The screenshot shows the EditPlus application window titled "Noname1.l - EditPlus". The menu bar includes File, Edit, View, Search, Document, Project, Tools, Browser, ZC, Window, and Help. The toolbar contains various icons for file operations and editing. On the left, the "Directory" pane shows the file explorer with the path [C:] Windows > Users > SRINIVASAMOOR > Desktop > csbs > CDT Lab > practice. The file list includes a.exe, arthlex.l, arthlex.l.bak, arthlyacc.tab.c, arthlyacc.y, arthlyacc.y.bak, arth.l, arth.l.bak, arthfinal.l, arthfinal.l.bak, arthfinaly.tab.c, arthfinaly.y, arthfinaly.y.bak, arthtest.l, arthtest.l.bak, arthtesty.tab.c, arthtesty.y, arthtesty.y.bak, arthy.tab.c, arthy.y, arthy.y.bak, and basics.exe. The main editor area contains the following C code:

```
1  #option noyywrap
2  %{
3      #include<stdio.h>
4  %}
5
6
7
8
9  int main()
10 {
11
12     return 0;
13 }
```

The status bar at the bottom indicates "In 13 col 2 13 00 PC ANSI".

To create a new yacc file file>new>yacc/bison then the file would be Noname2.y then we can save and use yacc file



The screenshot shows the EditPlus application window titled "Noname2.y - EditPlus". The menu bar and toolbar are the same as in the previous screenshot. The "Directory" pane shows the same file explorer path. The main editor area contains the following yacc code:

```
1  %{
2
3      #include<stdio.h>
4  %}
5
6  %token
7
8
9  %%
```

The status bar at the bottom indicates "In 9 col 3 9 00 PC ANSI".

Instructions to Run the Code :

LEX WORKING

```
pro1.1" [New] 0L, 0C written  
est69@hdc17001:~$ vim pro1.1
```

Vim filename.l

1. Press i in keyboard then start to type the code

Procedure :

2. After typing your code save as Press ESC :wq

3. lex filename.l or lex filename.c

4. cc lex.yy.c

5. ./a.out

YAAC Working

vim filename.l (For LEX CODE)

vim filename1.y (For YACC CODE)

lex filename.l

Yacc -d filename 1.y

Cc lex.yy.c y.tab.c

./a.out

Installing TDM-GCC Compile

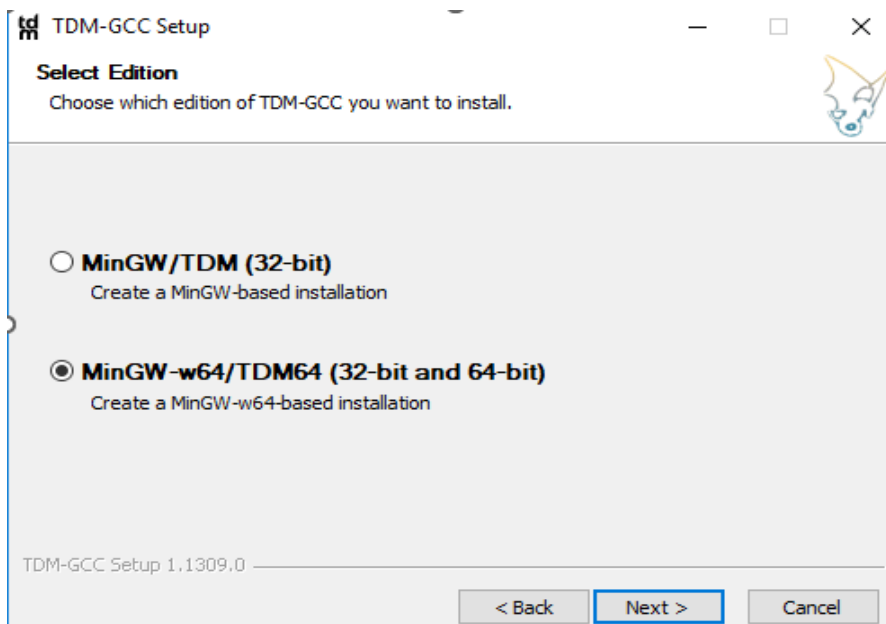
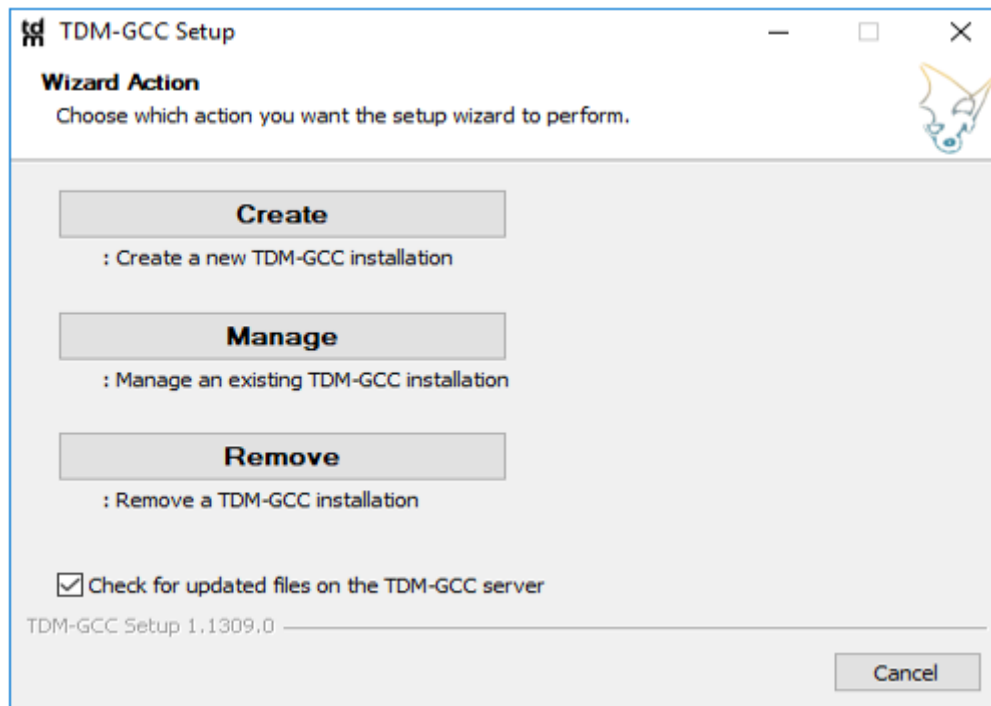
32-bit Version

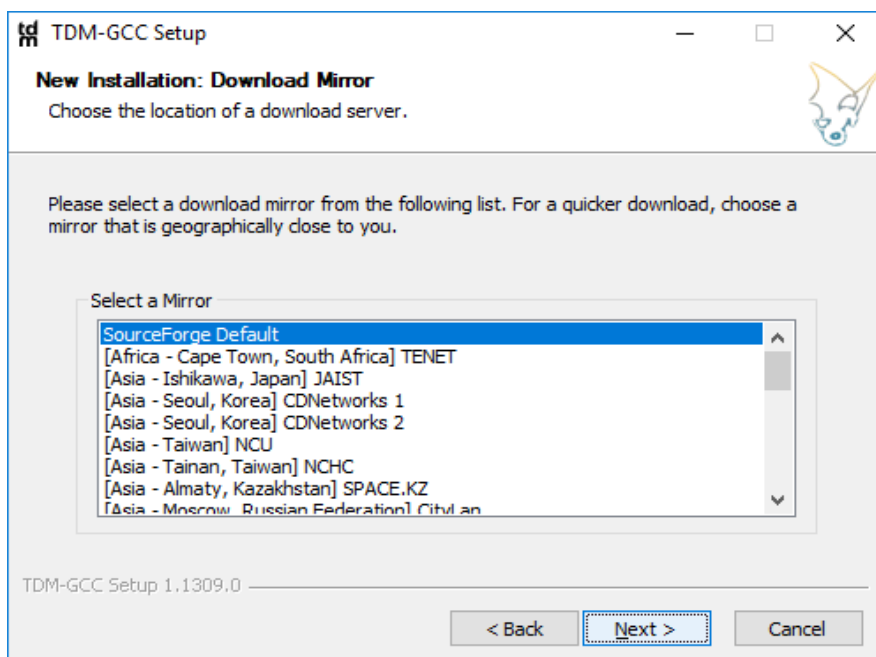
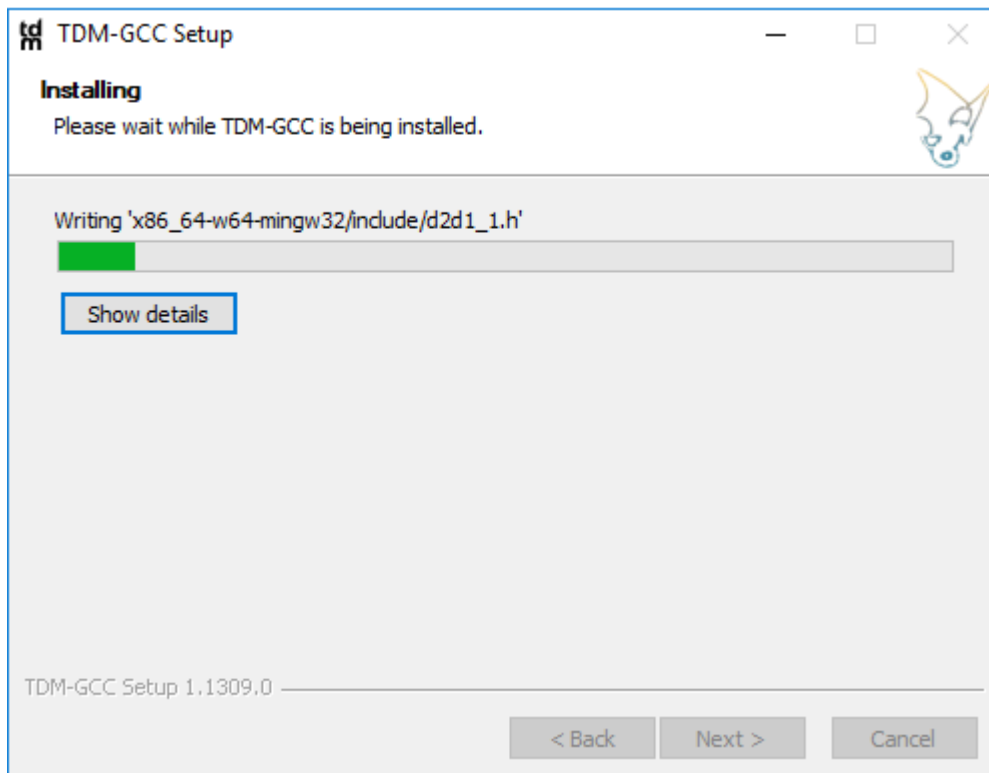
<http://sourceforge.net/projects/tdm-gcc/files/TDM-GCC%20Installer/tdm-gcc-5.1.0-3.exe/download>

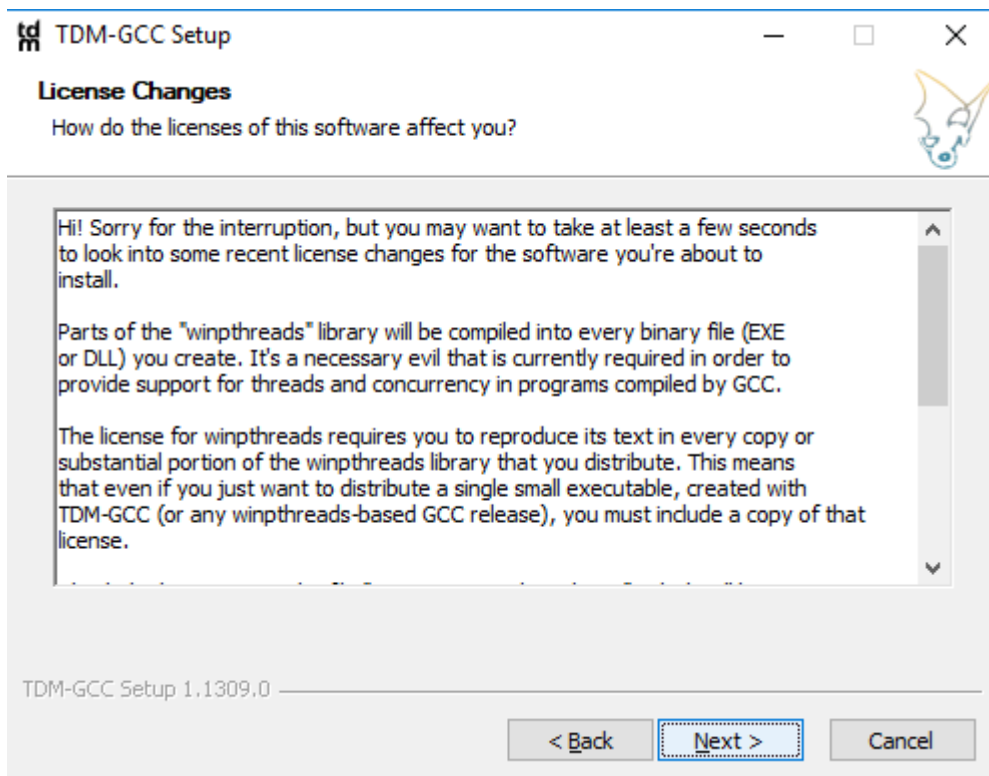
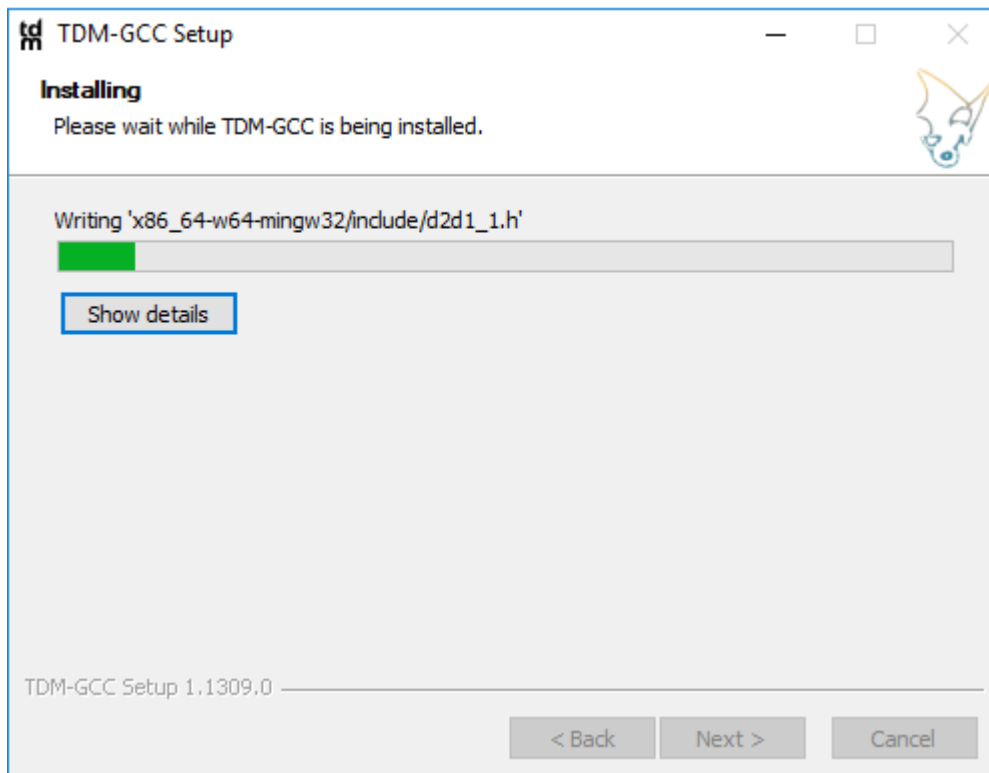
64-bit Version

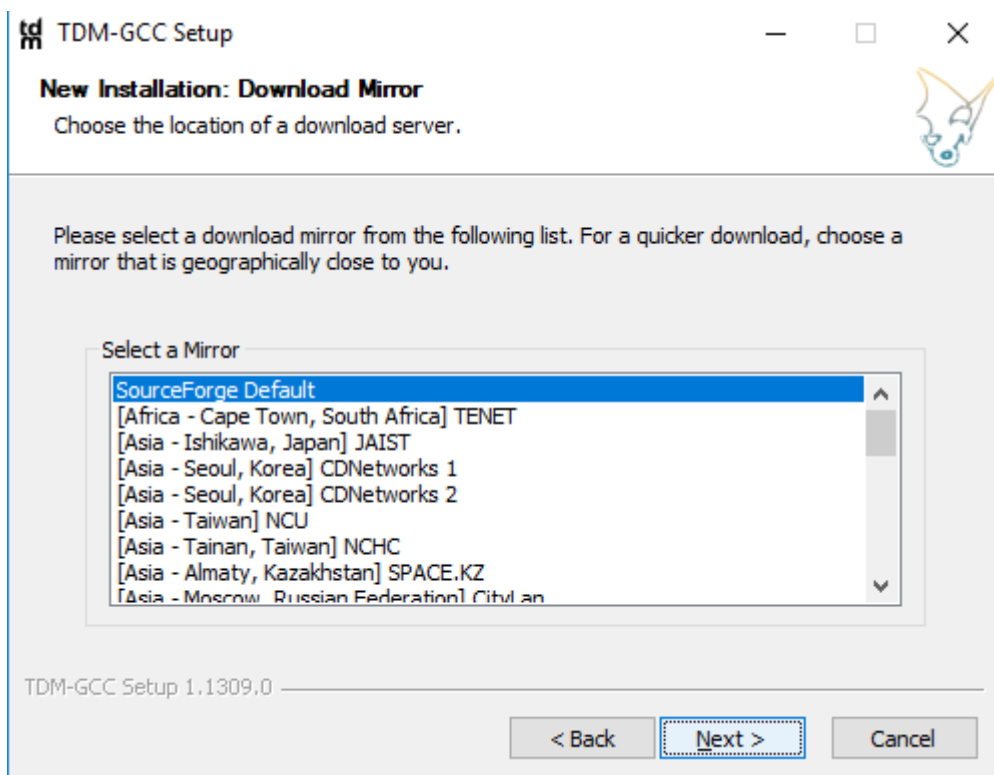
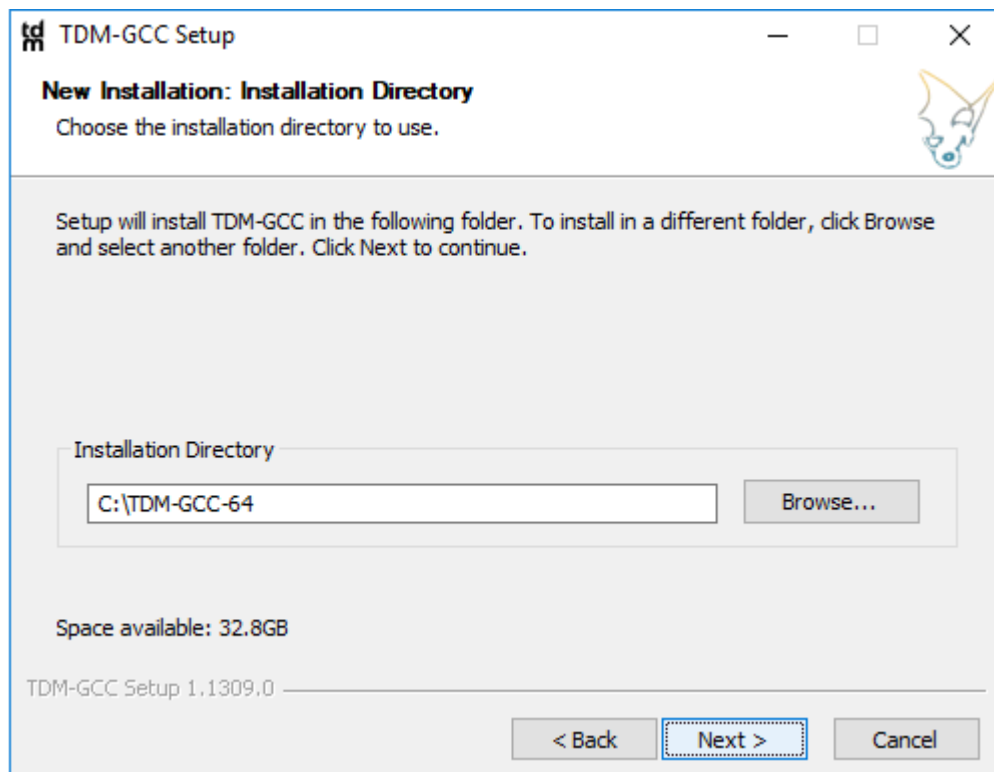
<http://sourceforge.net/projects/tdm-gcc/files/TDM-GCC%20Installer/tdm64-gcc-5.1.0-2.exe/download>

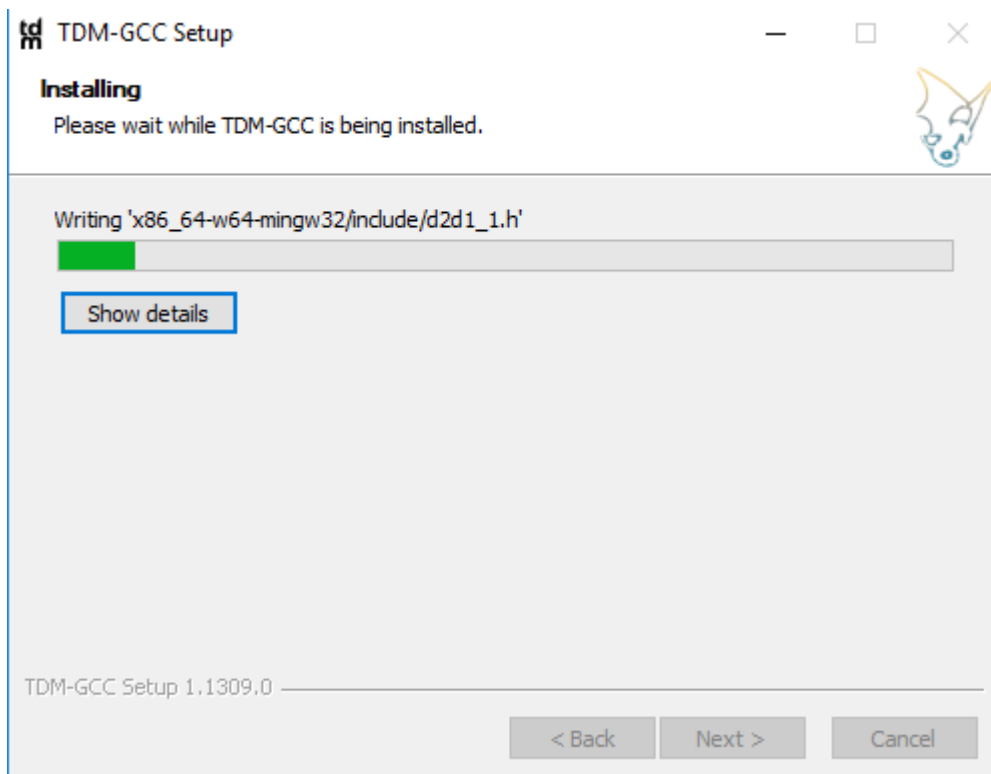
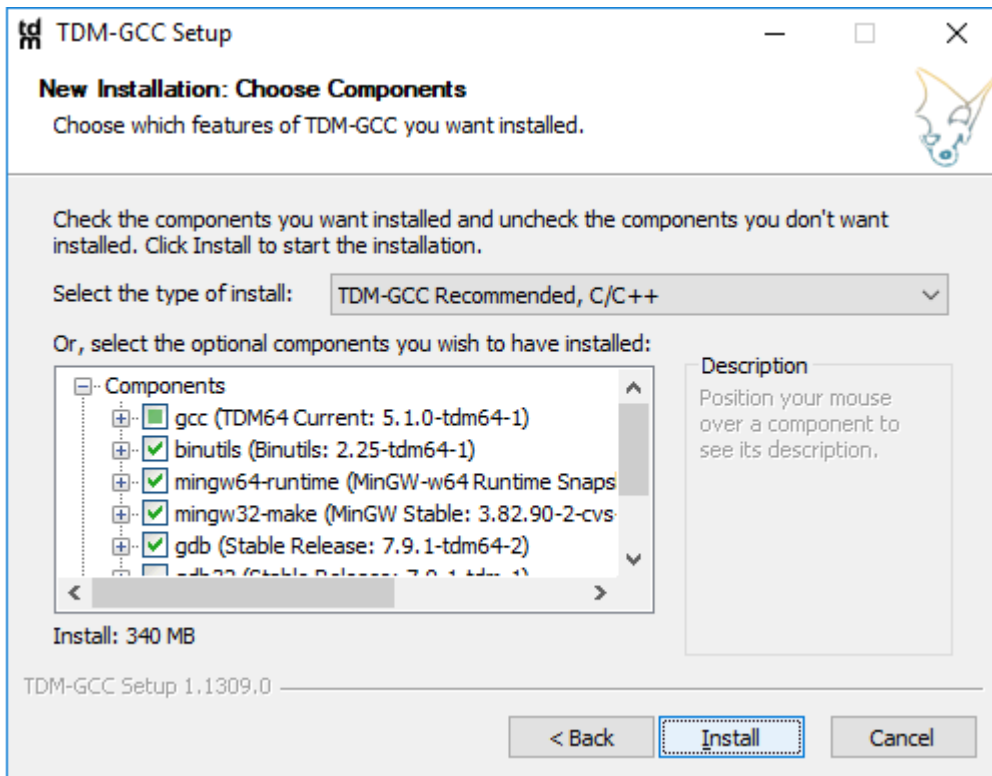
Double click the downloaded file and click Create.











Completing the TDM-GCC Setup Wizard

TDM-GCC has been installed on your computer.

Click Finish to close this wizard.



☐ Read the TDM-GCC README file (recommended)

< Back

Finish

Cancel

Installing Notepad

32-bit Version

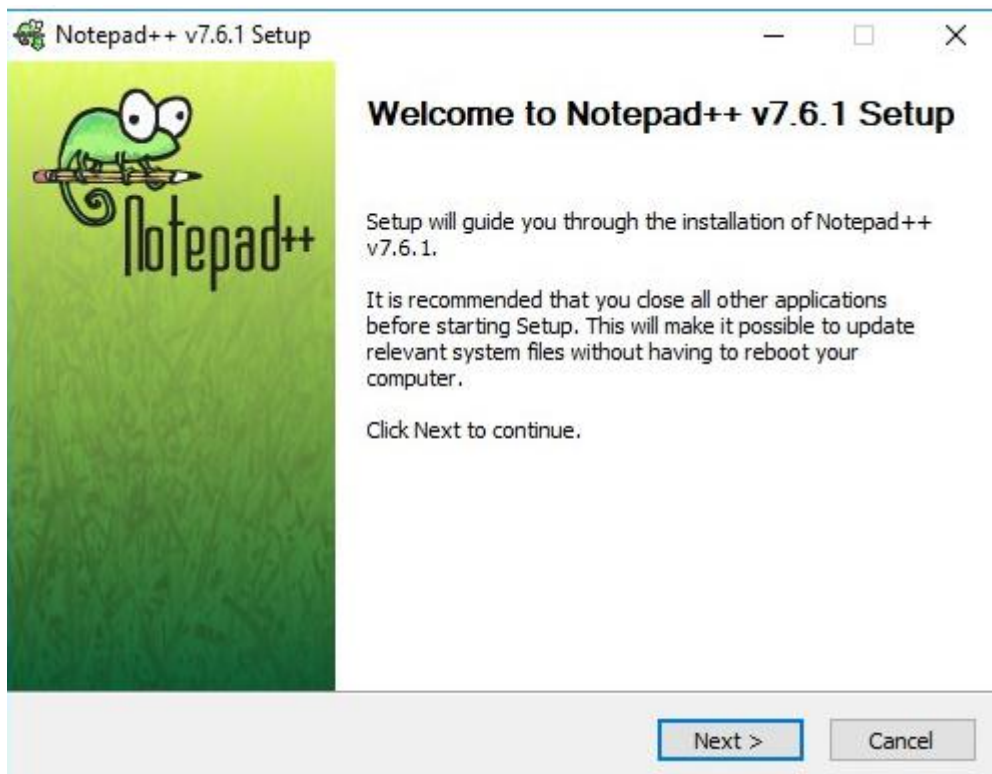
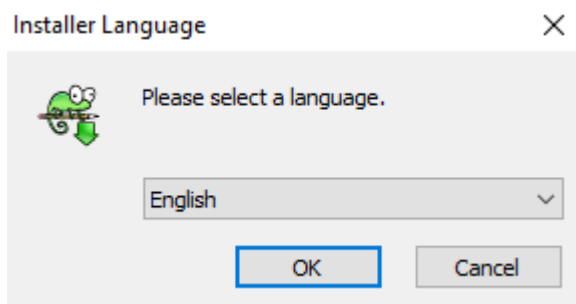
<https://notepad-plus-plus.org/repository/7.x/7.6.1/npp.7.6.1.Installer.exe>

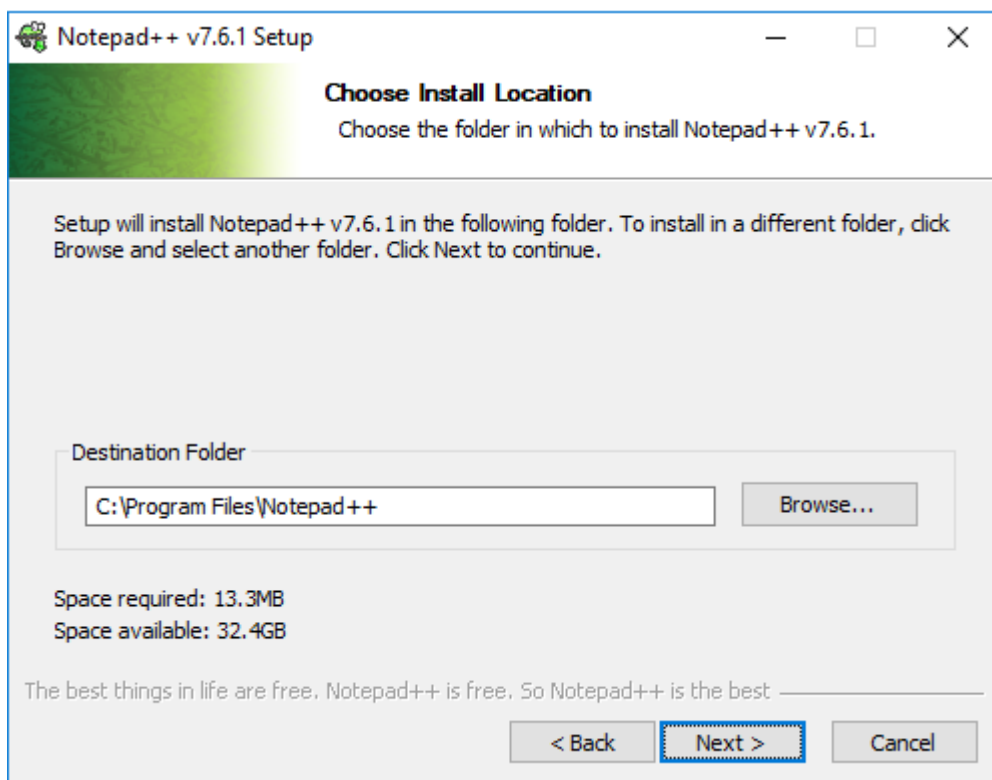
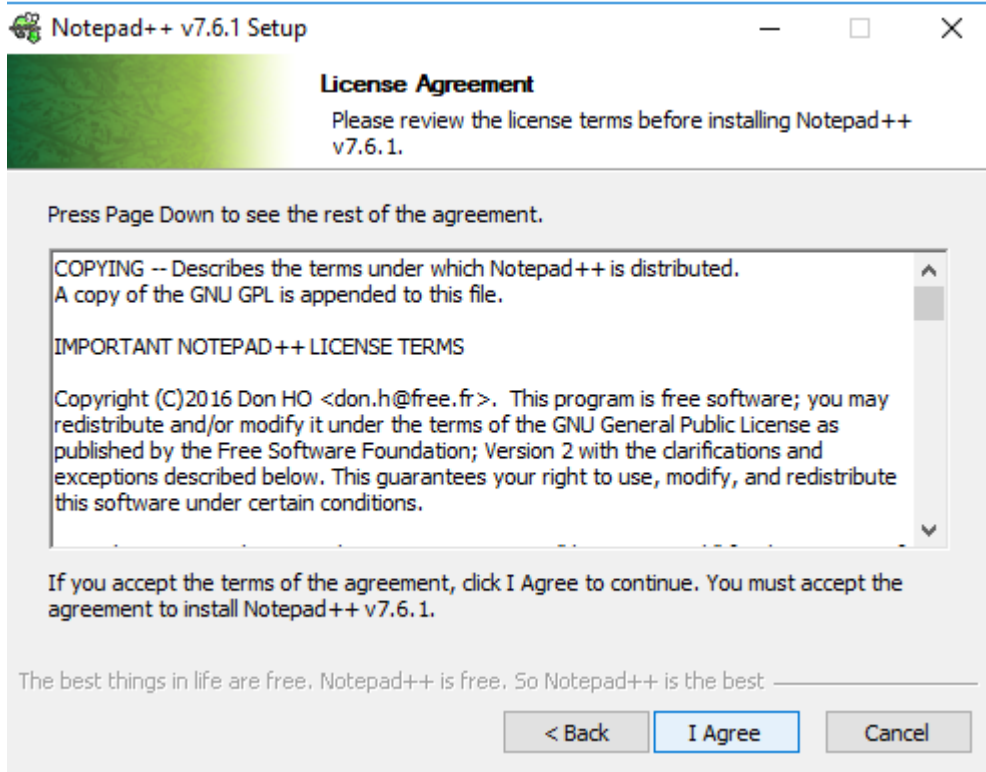
64-bit Version

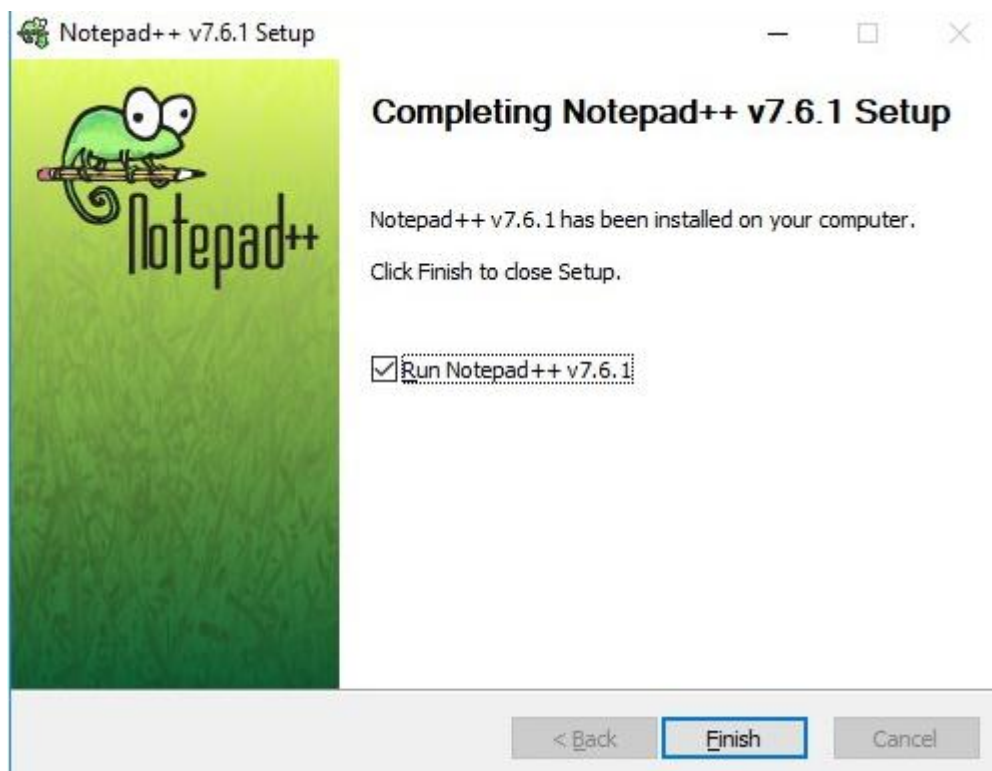
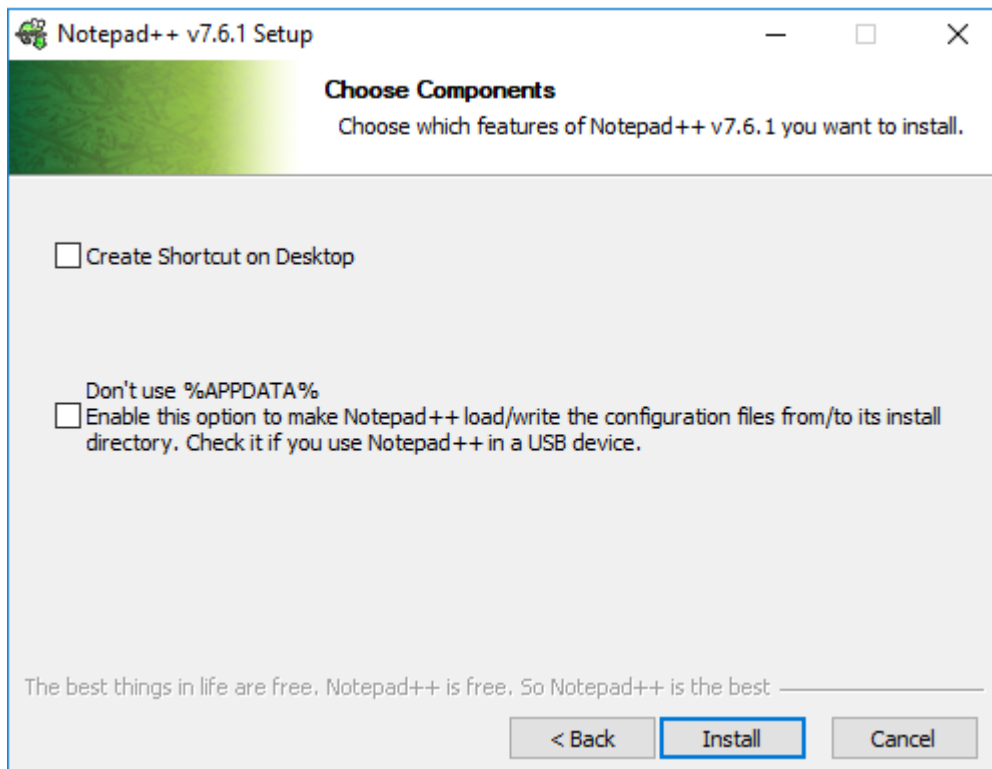
<https://notepad-plus-plus.org/repository/7.x/7.6.1/npp.7.6.1.Installer.x64.exe>

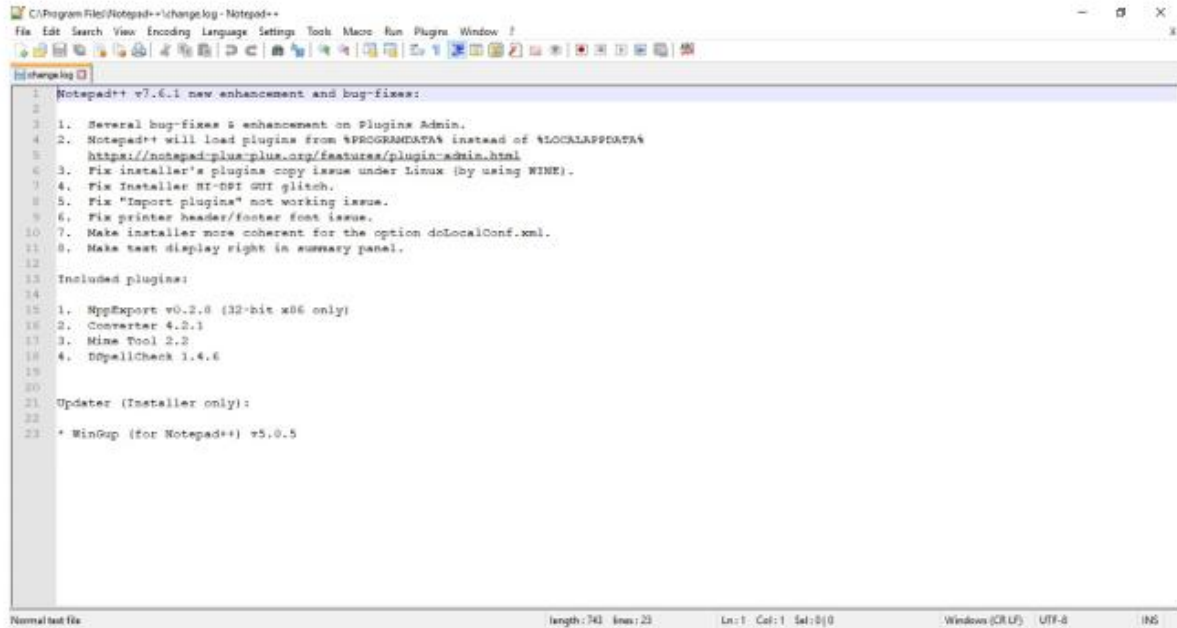
Double click the downloaded

file. Select a language.









```
1 Notepad++ v7.6.1 new enhancement and bug-fixes:
2
3 1. Several bug-fixes & enhancement on Plugins Admin.
4 2. Notepad++ will load plugins from %PROGRAMDATA% instead of %LOCALAPPDATA%
5   https://notepad-plus-plus.org/features/plugin-admin.html
6 3. Fix installer's plugins copy issue under Linux (by using WINE).
7 4. Fix Installer NT-OS! GUI glitch.
8 5. Fix "Import plugins" not working issue.
9 6. Fix printer header/footer font issue.
10 7. Make installer more coherent for the option doLocalConf.xml.
11 8. Make test display right in summary panel.
12
13 Included plugins:
14
15 1. NppExport v0.2.0 (32-bit x86 only)
16 2. Converter 4.2.1
17 3. Mine Tool 2.2
18 4. DSpellCheck 1.4.6
19
20
21 Updater (Installer only):
22
23 * WinGUp (for Notepad++) v5.0.5
```

Creating a Folder (Directory)

Press Window + R

key. Type the

command cmd.

